

GenAI Code Comparison Analysis

1. GenAI Programs Used

The two GenAI programs I used were ChatGPT and Gemini.

I accessed ChatGPT through the ChatGPT website by entering the assignment prompt into a new chat.

I accessed Gemini through Google's Gemini website by entering the same exact assignment prompt.

2. Prompt Used For Both GenAI Programs

Write a C++ object-oriented program for EECS 348 Assignment 3. The program should prioritize CEO emails using a MaxHeap as a priority queue. Do not use pre-existing heap modules or `std::priority_queue`. Implement the MaxHeap from scratch using a list-based implementation. Use objects/classes, not only standalone functions.

Input commands are:

EMAIL <sender category>,<subject line>,<date>

NEXT

READ

COUNT

Sender priority from highest to lowest:

Boss, Subordinate, Peer, ImportantPerson, OtherPerson.

If two emails have the same sender category, the newest date should be read first. Dates are in MM-DD-YYYY format.

NEXT should display the highest priority email without deleting it.

READ should remove the highest priority email.

COUNT should display the number of unread emails.

The program should handle empty queue cases.

Include clear comments and a prologue comment.

3. Generated Code From ChatGPT

```
/*
 * =====
 * Program: CEO Email Priority Queue
 * Course: EECS 348 - Assignment 3
 *
 * Description:
 * This program uses a MaxHeap priority queue to manage emails
 * for a CEO. The heap is implemented from scratch using a
 * vector/list-based structure. It does not use std::priority_queue
 * or any built-in heap functions.
 */
```

```

* Priority Rules:

* 1. Sender category priority:

*   Boss > Subordinate > Peer > ImportantPerson > OtherPerson

* 2. If sender categories are the same, the newest date has

*   higher priority.

* 3. If sender category and date are the same, the email that

*   arrived first has higher priority.

*

* Input Commands:

* EMAIL <sender category>,<subject line>,<date>

* NEXT

* READ

* COUNT

* =====

*/

```

```

#include <iostream>

```

```

#include <string>

```

```

#include <vector>

```

```

#include <sstream>

```

```

using namespace std;

```

```

class Date {

```

```

private:

```

```

    int month;

```

```

    int day;

```

```

    int year;

```

```

public:

```

```

    Date() {

```

```
    month = 0;

    day = 0;

    year = 0;
}
```

```
Date(string dateText) {

    char dash;

    stringstream ss(dateText);

    ss >> month >> dash >> day >> dash >> year;

}
```

```
bool isNewerThan(const Date& other) const {

    if (year != other.year) {

        return year > other.year;

    }

    if (month != other.month) {

        return month > other.month;

    }

    return day > other.day;

}
```

```
bool isEqualTo(const Date& other) const {

    return month == other.month && day == other.day && year == other.year;

}
```

```
string toString() const {

    string result = "";

    if (month < 10) {
```

```

        result += "0";
    }

    result += to_string(month);
    result += "-";

    if (day < 10) {
        result += "0";
    }

    result += to_string(day);
    result += "-";

    result += to_string(year);

    return result;
}
};

```

```

class Email {
private:
    string senderCategory;
    string subject;
    Date date;
    int arrivalOrder;

public:
    Email() {
        senderCategory = "";
        subject = "";
        arrivalOrder = 0;
    }
}

```

```
Email(string senderCategory, string subject, string dateText, int arrivalOrder)
```

```
    : date(dateText) {
```

```
    this->senderCategory = senderCategory;
```

```
    this->subject = subject;
```

```
    this->arrivalOrder = arrivalOrder;
```

```
}
```

```
int getSenderPriority() const {
```

```
    if (senderCategory == "Boss") {
```

```
        return 5;
```

```
    } else if (senderCategory == "Subordinate") {
```

```
        return 4;
```

```
    } else if (senderCategory == "Peer") {
```

```
        return 3;
```

```
    } else if (senderCategory == "ImportantPerson") {
```

```
        return 2;
```

```
    } else {
```

```
        return 1;
```

```
    }
```

```
}
```

```
bool hasHigherPriorityThan(const Email& other) const {
```

```
    if (getSenderPriority() != other.getSenderPriority()) {
```

```
        return getSenderPriority() > other.getSenderPriority();
```

```
    }
```

```
    if (!date.isEqualTo(other.date)) {
```

```
        return date.isNewerThan(other.date);
```

```
    }
```

```
    return arrivalOrder < other.arrivalOrder;
```

```

    }

    void print() const {
        cout << senderCategory << ", " << subject << ", " << date.toString() << endl;
    }
};

class MaxHeap {
private:
    vector<Email> heap;

    int parent(int index) const {
        return (index - 1) / 2;
    }

    int leftChild(int index) const {
        return (2 * index) + 1;
    }

    int rightChild(int index) const {
        return (2 * index) + 2;
    }

    void heapifyUp(int index) {
        while (index > 0 && heap[index].hasHigherPriorityThan(heap[parent(index)])) {
            swap(heap[index], heap[parent(index)]);
            index = parent(index);
        }
    }

    void heapifyDown(int index) {

```

```

        int largest = index;

        int left = leftChild(index);

        int right = rightChild(index);

        if (left < heap.size() && heap[left].hasHigherPriorityThan(heap[largest])) {
            largest = left;
        }

        if (right < heap.size() && heap[right].hasHigherPriorityThan(heap[largest])) {
            largest = right;
        }

        if (largest != index) {
            swap(heap[index], heap[largest]);
            heapifyDown(largest);
        }
    }

public:

    bool isEmpty() const {
        return heap.empty();
    }

    int count() const {
        return heap.size();
    }

    void insert(const Email& email) {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }

```

```

Email peek() const {
    return heap[0];
}

Email removeMax() {
    Email highestPriorityEmail = heap[0];

    heap[0] = heap.back();
    heap.pop_back();

    if (!heap.empty()) {
        heapifyDown(0);
    }

    return highestPriorityEmail;
}
};

int main() {
    MaxHeap emailQueue;

    string commandLine;

    int arrivalCounter = 0;

    while (getline(cin, commandLine)) {
        if (commandLine.substr(0, 5) == "EMAIL") {
            string emailData = commandLine.substr(6);

            string senderCategory;

            string subject;

            string dateText;

```

```

        stringstream ss(emailData);

        getline(ss, senderCategory, ',');
        getline(ss, subject, ',');
        getline(ss, dateText);

        Email newEmail(senderCategory, subject, dateText, arrivalCounter);
        emailQueue.insert(newEmail);

        arrivalCounter++;
    } else if (commandLine == "NEXT") {
        if (emailQueue.isEmpty()) {
            cout << "No emails to read." << endl;
        } else {
            emailQueue.peek().print();
        }
    } else if (commandLine == "READ") {
        if (emailQueue.isEmpty()) {
            cout << "No emails to read." << endl;
        } else {
            emailQueue.removeMax().print();
        }
    } else if (commandLine == "COUNT") {
        cout << emailQueue.count() << endl;
    }
}

return 0;
}

```

4. Generated Code From Gemini

/*

```
* Course: EECS 348

* Assignment: EECS 348 Assignment 3 - Priority Queue Implementation

* File: main.cpp

* Description: Object-oriented C++ program that prioritizes CEO emails using

*             a MaxHeap-based Priority Queue built from scratch without standard

*             heap libraries. Prioritizes by Sender Category first, then by Date (newest first).

*/
```

```
#include
```

```
#include
```

```
#include
```

```
// Enumeration for Sender Categories to easily compare priority levels
```

```
enum SenderCategory {
```

```
    OTHER_PERSON = 0,
```

```
    IMPORTANT_PERSON = 1,
```

```
    PEER = 2,
```

```
    SUBORDINATE = 3,
```

```
    BOSS = 4
```

```
};
```

```
// Helper function to map category strings to their priority enum values
```

```
SenderCategory parseCategory(const std::string& catStr) {
```

```
    if (catStr == "Boss") return BOSS;
```

```
    if (catStr == "Subordinate") return SUBORDINATE;
```

```
    if (catStr == "Peer") return PEER;
```

```
    if (catStr == "ImportantPerson") return IMPORTANT_PERSON;
```

```
    return OTHER_PERSON;
```

```
}
```

```
// Class representing a Date in MM-DD-YYYY format
```

```

class Date {
private:
    int month;

    int day;

    int year;

public:
    Date() : month(0), day(0), year(0) {}

    // Parses string formatted as "MM-DD-YYYY"
    Date(const std::string& dateStr) {
        std::stringstream ss(dateStr);

        std::string token;

        std::getline(ss, token, '-');
        month = std::stoi(token);

        std::getline(ss, token, '-');
        day = std::stoi(token);

        std::getline(ss, token, '-');
        year = std::stoi(token);
    }

    // Returns true if this date is newer than the other date
    bool isNewerThan(const Date& other) const {
        if (year != other.year) return year > other.year;

        if (month != other.month) return month > other.month;

        return day > other.day;
    }
}

```

```

// Returns true if two dates are identical

bool equals(const Date& other) const {

    return (year == other.year && month == other.month && day == other.day);

}

// Converts date back to standard string format

std::string toString() const {

    std::string m = (month < 10 ? "0" : "") + std::to_string(month);

    std::string d = (day < 10 ? "0" : "") + std::to_string(day);

    return m + "-" + d + "-" + std::to_string(year);

}

};

// Class representing an Email object

class Email {

private:

    std::string senderStr;

    SenderCategory category;

    std::string subject;

    Date date;

public:

    Email() : senderStr(""), category(OTHER_PERSON), subject(""), date() {}

    Email(const std::string& sender, const std::string& sub, const std::string& dateStr)

        : senderStr(sender), category(parseCategory(sender)), subject(sub), date(dateStr) {}

    SenderCategory getCategory() const { return category; }

    std::string getSenderStr() const { return senderStr; }

    std::string getSubject() const { return subject; }

    Date getDate() const { return date; }

```

```

// Determines if this email has higher priority than another
// Priority Rule 1: Higher sender category rank
// Priority Rule 2: Newer date if categories are equal
bool hasHigherPriorityThan(const Email& other) const {
    if (category != other.category) {
        return category > other.category;
    }
    return date.isNewerThan(other.date);
}

void print() const {
    std::cout << senderStr << ", " << subject << ", " << date.toString() << std::endl;
}
};

```

// Node structure for dynamically linked array/list implementation of MaxHeap

```

class MaxHeap {
private:
    Email* heapArray;
    int capacity;
    int size;

    // Resizes internal array dynamically when full
    void resize() {
        capacity *= 2;
        Email* newArray = new Email[capacity];
        for (int i = 0; i < size; ++i) {
            newArray[i] = heapArray[i];
        }
        delete[] heapArray;
    }
};

```

```

    heapArray = newArray;
}

// Restores heap property upward from index
void heapifyUp(int index) {
    while (index > 0) {
        int parentIndex = (index - 1) / 2;
        if (heapArray[index].hasHigherPriorityThan(heapArray[parentIndex])) {
            std::swap(heapArray[index], heapArray[parentIndex]);
            index = parentIndex;
        } else {
            break;
        }
    }
}

// Restores heap property downward from index
void heapifyDown(int index) {
    while (true) {
        int leftChild = 2 * index + 1;
        int rightChild = 2 * index + 2;
        int highest = index;

        if (leftChild < size && heapArray[leftChild].hasHigherPriorityThan(heapArray[highest])) {
            highest = leftChild;
        }

        if (rightChild < size && heapArray[rightChild].hasHigherPriorityThan(heapArray[highest])) {
            highest = rightChild;
        }

        if (highest != index) {

```

```

        std::swap(heapArray[index], heapArray[highest]);

        index = highest;
    } else {
        break;
    }
}
}

```

public:

```

MaxHeap(int initialCapacity = 10) : capacity(initialCapacity), size(0) {
    heapArray = new Email[capacity];
}

```

```

~MaxHeap() {
    delete[] heapArray;
}

```

// Inserts a new email into the priority queue

```

void insert(const Email& email) {
    if (size == capacity) {
        resize();
    }

    heapArray[size] = email;

    heapifyUp(size);

    size++;
}

```

// Displays the highest priority email without removing it

```

void peekNext() const {
    if (isEmpty()) {
        std::cout << "Queue is empty. No emails to display." << std::endl;
    }
}

```

```

        return;
    }

    heapArray[0].print();
}

// Removes and displays the highest priority email
void extractMax() {
    if (isEmpty()) {
        std::cout << "Queue is empty. No emails to read." << std::endl;
        return;
    }
    heapArray[0].print();
    heapArray[0] = heapArray[size - 1];
    size--;
    if (size > 0) {
        heapifyDown(0);
    }
}

int getSize() const {
    return size;
}

bool isEmpty() const {
    return size == 0;
}
};

// Class to manage commands and interact with the user/file input
class EmailPriorityQueueApp {
private:

```

```
MaxHeap emailQueue;
```

```
// Helper to parse comma-delimited parameters for EMAIL command
```

```
void parseAndAddEmail(const std::string& args) {  
    std::stringstream ss(args);  
    std::string sender, subject, dateStr;  
  
    if (std::getline(ss, sender, ',') &&  
        std::getline(ss, subject, ',') &&  
        std::getline(ss, dateStr)) {  
  
        // Trim leading spaces from parameters if present  
        if (!sender.empty() && sender[0] == ' ') sender.erase(0, 1);  
        if (!subject.empty() && subject[0] == ' ') subject.erase(0, 1);  
        if (!dateStr.empty() && dateStr[0] == ' ') dateStr.erase(0, 1);  
  
        Email newEmail(sender, subject, dateStr);  
        emailQueue.insert(newEmail);  
    }  
}
```

```
public:
```

```
void run() {  
    std::string line;  
    while (std::getline(std::cin, line)) {  
        if (line.empty()) continue;  
  
        if (line.rfind("EMAIL", 0) == 0) {  
            // Extracts arguments after "EMAIL "  
            std::string args = line.substr(6);  
            parseAndAddEmail(args);  
        }  
    }  
}
```

```

        } else if (line == "NEXT") {
            emailQueue.peekNext();
        } else if (line == "READ") {
            emailQueue.extractMax();
        } else if (line == "COUNT") {
            std::cout << "Unread emails: " << emailQueue.getSize() << std::endl;
        }
    }
}

};

int main() {
    EmailPriorityQueueApp app;

    app.run();

    return 0;
}

```

5. Code Analysis

The ChatGPT code had a better start because it used Date, Email, and MaxHeap classes. It also used a vector/list-based heap and did not use `std::priority_queue`. The priority logic was mostly correct because it compared sender category first, then date, then arrival order.

However, the ChatGPT code did not fully match the required output format. NEXT printed the email in one line instead of using Sender, Subject, and Date labels. READ also printed the email, but according to the assignment READ should only remove the email. COUNT printed only the number instead of the full sentence.

The Gemini code also used classes and a MaxHeap idea, but the pasted code had incomplete include lines, so it would not compile as written. It also did not handle the arrival-order tie breaker when two emails had the same sender category and same date. Its NEXT, READ, and COUNT outputs also did not match the assignment format.

For execution time, both programs were trying to use a heap, so insert and remove operations are $O(\log n)$, NEXT is $O(1)$, and COUNT is $O(1)$. For space complexity, both use $O(n)$ space because all unread emails are stored in the heap.

6. Selected Baseline Program

I selected the ChatGPT code as the baseline because it was closer to the assignment requirements. It already had the arrival-order tie breaker, which is important when two emails have the same sender category and date. Gemini had a useful structure, but it had missing include statements and did not fully handle the tie case.

7. Improvements Made To The Final Program

I improved the final program by fixing the output format for NEXT, READ, and COUNT. NEXT now displays the email using Sender, Subject, and Date labels. READ now removes the highest-priority email without printing it. COUNT now prints the full sentence required by the sample output.

I also kept the MaxHeap implementation from scratch and did not use `std::priority_queue`. I kept the sender priority rule, newest date rule, and arrival-order tie breaker. I added empty queue handling so NEXT and READ do not crash when there are no emails. I also added a complete prologue comment and more detailed comments to match the rubric.

8. Final Conclusion

Both GenAI programs gave me a useful starting point, but neither raw program fully satisfied the assignment requirements. The main issues were incorrect output formatting, READ printing the email when it should only remove it, incomplete comments, and in Gemini's case, missing include statements and no arrival-order tie breaker.

For my final version, I used the ChatGPT code as the baseline because it was closer to the requirements. I improved it by fixing the NEXT, READ, and COUNT behavior, keeping the MaxHeap implementation from scratch, adding empty queue handling, and making the comments and prologue more complete. The final program follows the required priority rules, uses objects/classes, does not use `std::priority_queue`, and matches the sample output format.